

# Can General-Purpose Large Language Models Detect Algorithm Debt in Machine Learning Code? A Comparative Study Against Rule-Based Detection

Emaan Arif Khan, Fatima Azfar

Department of Computer Science and Information Technology  
NED University of Engineering and Technology  
Karachi, Pakistan

**Abstract**—Machine learning (ML) systems accumulate a distinct form of technical debt, referred to as algorithm debt, through recurring implementation shortcuts such as hardcoded hyperparameters, missing data validation, train/test leakage, absent reproducibility controls, and silently suppressed exceptions. Detecting these anti-patterns currently relies on specialized static-analysis tooling or manual code review. This study investigates whether a general-purpose large language model (LLM), without task-specific fine-tuning, can detect five such anti-patterns in real-world ML code with accuracy comparable to a purpose-built rule-based detector. We constructed an abstract-syntax-tree (AST) based rule detector targeting the five anti-patterns and applied it to 33 public GitHub repositories comprising 1,619 Python files as a baseline. We then prompted an LLM (Llama-3.3-70B, served via Groq) to classify a stratified sample of 40 files for the same anti-patterns and computed precision, recall, and F1-score per category against the rule-based baseline. Results show the LLM achieved strong agreement on anti-patterns with clear, local syntactic signatures — hardcoded hyperparameters ( $F1 = 0.83$ ) and silent exception handling ( $F1 = 0.83$ ) — but performed poorly on categories requiring broader file-level context, particularly missing data validation ( $F1 = 0.00$ ) and reproducibility control ( $F1 = 0.23$ , precision = 0.13 despite recall = 1.00). Manual inspection of disagreements revealed the LLM systematically over-flags these two categories on files where they do not structurally apply, such as vendored libraries and model-architecture definitions. We conclude that general-purpose LLMs are currently more suited to high-recall triage of syntactically local algorithm debt than as standalone precise detectors, and that file-role classification may be a necessary pre-filtering step for future LLM-assisted debt detection.

**Keywords**—algorithm debt, technical debt, large language models, machine learning engineering, static analysis, code smell detection, reproducibility

## I. INTRODUCTION

As machine learning systems move from research prototypes into production pipelines, they accumulate a form of technical debt distinct from traditional software debt. Sculley et al. [1] first characterized this “hidden technical debt” in ML systems, noting that shortcuts taken during rapid model development — undocumented configuration, entangled data dependencies, and unstable training procedures — compound over time into costly maintenance burdens. We refer to the code-level manifestation of this debt as algorithm debt: concrete, identifiable anti-patterns in ML source code that increase the risk of silent failures, irreproducible results, and degraded model reliability.

Detecting algorithm debt currently depends on specialized static-analysis tooling or manual code review, both of which require domain expertise and do not scale easily across the rapidly growing volume of ML code in open-source and industrial repositories. Meanwhile, general-purpose large language models (LLMs) are increasingly used for code review, bug detection, and code-smell identification [2]–[4], raising a natural question: can a general-purpose LLM, without any task-specific fine-tuning, detect ML-specific algorithm debt with accuracy comparable to a dedicated detection tool?

This paper investigates that question empirically. We define five concrete, well-scoped algorithm-debt anti-patterns, build a rule-based AST detector to serve as a baseline, and evaluate an LLM’s ability to detect the same

anti-patterns on real-world open-source ML code. Our contributions are as follows: (1) a lightweight, reproducible rule-based detector for five ML algorithm-debt anti-patterns; (2) an empirical comparison of LLM-based versus rule-based detection across 1,619 files from 33 public repositories; and (3) a qualitative analysis of systematic disagreement patterns that reveals specific failure modes of LLM-based detection.

## II. RELATED WORK

### A. Technical and Algorithm Debt in ML Systems

Sculley et al. [1] introduced the concept of hidden technical debt in ML systems, identifying boundary erosion, entanglement, and configuration debt as recurring sources of long-term maintenance cost. Building on this foundation, Suominen and Hettiarachchi et al. [2] proposed a taxonomy of algorithm debt specific to machine and deep learning systems, surveying the landscape of debt categories and the tooling gap that remains for automated, scalable detection. Our five anti-patterns are drawn directly from categories described in this taxonomy.

### B. LLMs for Code Smell and Bug Detection

A growing body of work has evaluated LLMs on general software-engineering detection tasks. Comparative studies of prompt-based LLMs against traditional heuristic and machine-learning-based code-smell detectors on Java and mixed-language datasets report that LLMs perform competitively on structurally simple smells but degrade on smells requiring broader design-level reasoning [3], [4]. Related work applying LLMs to test-smell identification

across multiple codebases reports similarly uneven performance across smell categories [5]. Studies benchmarking LLMs against traditional static-analysis tools for bug and vulnerability detection report that LLMs can match or exceed rule-based tools on recall, particularly for well-scoped, syntactically identifiable issues, while precision remains a persistent weakness [6], [7], [8].

### C. Data Leakage and Reproducibility Tooling

Independent of LLM-based approaches, dedicated static-analysis tools have been developed to detect data leakage in ML pipelines, including IDE plugins that identify overlap, preprocessing, and multi-test leakage in Python and Jupyter Notebook code [9], [10], building on earlier static detection work for notebook environments [11]. Separately, a substantial body of work documents a broader reproducibility crisis in ML-driven research, attributing much of it to leakage, undocumented training conditions, and inconsistent reporting practices [12], [13], [14], with recent work extending this concern specifically to LLM-based software-engineering research artifacts [15].

Our work sits at the intersection of these three threads: rather than evaluating LLMs on general code smells or building a dedicated leakage-detection tool, we narrow the scope specifically to a taxonomy-grounded set of ML algorithm-debt anti-patterns and directly compare zero-shot LLM detection against a purpose-built rule-based baseline, a combination that remains comparatively unexamined in the current literature.

## III. METHODOLOGY

### A. Anti-Pattern Definitions

We selected five algorithm-debt anti-patterns based on their concreteness, prevalence in ML codebases, and suitability for unambiguous ground-truth labeling: (1) hardcoded hyperparameters — learning rate, batch size, epoch count, or similar values written as literals rather than configurable parameters; (2) missing data validation — absence of checks for missing values, invalid dtypes, or malformed schema prior to model fitting; (3) train/test leakage — a transformer’s fit or fit\_transform method invoked on the full dataset prior to a train/test split; (4) no reproducibility control — absence of a fixed random seed across the training pipeline; and (5) silent exception handling — bare except clauses or exception blocks whose only content is pass or continue.

### B. Rule-Based Baseline Detector

We implemented an AST-based static analyzer in Python using the built-in ast module. The detector parses each file’s syntax tree and applies pattern-specific heuristics: literal-valued assignments and keyword arguments matching known hyperparameter names; call-order analysis to detect fit/fit\_transform invocations preceding train\_test\_split; presence or absence of validation-suggestive calls (e.g., isna, dropna, assert) in files that combine pandas usage with model fitting; presence or absence of seed-setting calls or random\_state keyword arguments; and detection of bare or

trivial except handlers. This detector serves as the ground-truth baseline against which LLM output is compared.

### C. Corpus Collection

We collected 33 public repositories from GitHub via the GitHub Search API, using queries targeting small-to-medium ML training pipelines (e.g., “sklearn train\_test\_split,” “pytorch training loop,” “kaggle competition solution”), restricted to repositories with 5–200 stars to favor individual or small-team projects over large frameworks. Repositories were shallow-cloned, yielding 1,619 Python files. The rule-based detector was applied to the full corpus to establish baseline anti-pattern prevalence.

### D. LLM Evaluation

From the full corpus, we drew a stratified sample of 40 files, weighted toward files the rule-based detector had flagged (60%) with the remainder drawn from unflagged files, to ensure the evaluation set contained both positive and negative cases for every anti-pattern. Each file was submitted independently to Llama-3.3-70B-Versatile (served via the Groq API) with a system prompt defining all five anti-patterns and instructing the model to return a structured JSON classification (true/false) for each. Temperature was set to 0 to maximize determinism. LLM output was parsed and compared directly against the rule-based baseline for the same files, treating the rule-based result as ground truth.

## IV. RESULTS

Table I reports precision, recall, and F1-score for the LLM’s detection of each anti-pattern against the rule-based baseline across the 40-file evaluation sample.

**TABLE I. PRECISION, RECALL, AND F1-SCORE OF LLM DETECTION AGAINST RULE-BASED BASELINE (n = 40 FILES)**

| Anti-Pattern               | Precision | Recall | F1   |
|----------------------------|-----------|--------|------|
| Hardcoded hyperparameters  | 0.83      | 0.83   | 0.83 |
| Missing data validation    | 0.00      | 0.00   | 0.00 |
| Train/test leakage         | N/A*      | N/A*   | N/A* |
| No reproducibility control | 0.13      | 1.00   | 0.23 |
| Silent exception handling  | 0.83      | 0.83   | 0.83 |

\*No instances of train/test leakage occurred in the 40-file sample, consistent with its low base rate (0.4%) across the full 1,619-file corpus; this category could not be evaluated at the current sample size.

The LLM achieved strong, near-identical agreement with the rule-based baseline on hardcoded hyperparameters and silent exception handling (F1 = 0.83 for both), both anti-patterns with clear, syntactically local signatures — a literal value assigned to a recognizable variable name, or a bare except block. Performance was markedly weaker on missing data validation (F1 = 0.00, 33 false positives out of 40 files) and no reproducibility control (F1 = 0.23; recall = 1.00 but precision = 0.13, 27 false positives). Train/test leakage did

not occur in the 40-file sample, consistent with its low prevalence (0.4%, 7/1,619 files) in the full corpus, and could not be evaluated at this sample size.

## V. DISCUSSION

Manual inspection of the 68 recorded disagreements reveals a consistent pattern: the overwhelming majority (60 of 68) are false positives on missing data validation and no reproducibility control, and they cluster on files that are not, in fact, ML training pipelines. Representative examples include a vendored copy of the Python requests HTTP library flagged for both missing data validation and no reproducibility control despite containing no model-fitting code; a model-architecture definition file (`deepseek_v3/__init__.py`) flagged on the same two categories despite defining structure rather than executing training; and a model-weight download utility (`download_hf_assets.py`) flagged despite performing no data validation-relevant operations at all.

This suggests the LLM is not reasoning about whether a given file's role in the pipeline makes a given anti-pattern applicable at all. Instead, it appears to apply a coarse heuristic — treating the presence of ML-related imports or code style as sufficient grounds to flag missing validation or reproducibility controls, regardless of whether the file performs data loading, model fitting, or neither. This produces the observed pattern of near-perfect recall (the LLM rarely misses a true positive because it flags almost everything) paired with very low precision (most of its positive flags are spurious).

By contrast, hardcoded hyperparameters and silent exception handling are detectable from a small, local code span — a single assignment statement or except block — without requiring the model to infer the broader role of the file. This distinction offers a practical heuristic for where LLM-based algorithm-debt detection is currently reliable: anti-patterns with local, self-contained syntactic signatures are well-suited to zero-shot LLM detection, while anti-patterns requiring file-level or pipeline-level context require either more sophisticated prompting strategies, retrieval of surrounding project context, or a file-role classification step performed prior to anti-pattern detection.

## VI. THREATS TO VALIDITY

Several limitations should be considered when interpreting these results. First, the evaluation sample (40 files) is modest, and train/test leakage could not be evaluated due to its low base rate; a larger sample stratified specifically to guarantee minimum representation of rare categories would strengthen future work. Second, the rule-based detector, while carefully validated on synthetic test cases, is itself a heuristic and may not represent a perfect ground truth — some of the LLM's disagreements may reflect legitimate detections the rule-based tool missed rather than LLM error, a possibility partially supported by our qualitative review but not exhaustively verified across all 68 disagreements. Third, results are based on a single LLM (Llama-3.3-70B) and

prompting strategy; performance may vary across other general-purpose models and with more sophisticated prompting techniques such as chain-of-thought reasoning or few-shot examples.

## VII. CONCLUSION

This study provides an initial empirical answer to whether general-purpose LLMs can detect ML algorithm debt without task-specific fine-tuning: LLMs perform well on anti-patterns with clear, local syntactic signatures, but currently over-generalize on anti-patterns requiring judgment about a file's role within the broader ML pipeline, producing high recall at the cost of precision. These findings suggest that LLM-assisted algorithm-debt detection is currently best suited to high-recall triage of syntactically identifiable issues rather than standalone precise detection, and that file-role classification represents a promising direction for improving LLM-based detection of context-dependent anti-patterns in future work.

## REFERENCES

- [1] D. Sculley et al., “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems* (NeurIPS), 2015, pp. 2503–2511.
- [2] H. Suominen, C. Hettiarachchi et al., “A survey of algorithm debt in machine and deep learning systems,” *ACM Computing Surveys*, 2026.
- [3] “Beyond strict rules: Assessing the effectiveness of large language models for code smell detection,” arXiv:2601.09873, 2026.
- [4] “Leveraging prompt-based large language models for code smell detection: A comparative study on the MLCQ dataset,” in *Proc. Int. Conf. on Software Engineering and Knowledge Engineering*, Springer, 2025.
- [5] “Attentionsmelling: Using large language models to identify code smells,” 2025.
- [6] “LLM-GUARD: Large language model-based detection and repair of bugs and security vulnerabilities in C++ and Python,” arXiv:2508.16419, 2026.
- [7] “Large language models versus static code analysis tools: A systematic benchmark for vulnerability detection,” arXiv:2508.04448, 2025.
- [8] “Augmenting large language models with static code analysis for automated code quality improvements,” arXiv:2506.10330, 2026.
- [9] E. A. AlOmar, C. DeMario, R. Shagawat, and B. Kreiser, “LeakageDetector: An open source data leakage analysis tool in machine learning pipelines,” arXiv:2503.14723, 2025, pp. 844–849.
- [10] O. Truong et al., “LeakageDetector 2.0: Analyzing data leakage in Jupyter-driven machine learning pipelines,” arXiv:2509.15971, 2025.
- [11] C. Yang, R. A. Brower-Sinning, G. Lewis, and C. Kästner, “Data leakage in notebooks: Static detection and better processes,” in *Proc. 37th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2022, pp. 1–12.

- [12] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, 2023, Art. no. 100804.
- [13] H. Semmelrock, S. Kopeinik, D. Theiler, T. Ross-Hellauer, and D. Kowald, “Reproducibility in machine learning-driven research,” *arXiv:2307.10320*, 2023.
- [14] H. Semmelrock et al., “Reproducibility in machine learning-based research: Overview, barriers and drivers,” *arXiv:2406.14325*, 2025.
- [15] “Large language models for software engineering: A reproducibility crisis,” *arXiv:2512.00651*, 2026.